

Programação Dinâmica

Prof. Leandro I. Pinto

Programação Dinâmica

- Resolve problemas combinando as soluções para subproblemas
- Se aplica quando subproblemas se sobrepõem
 - Subproblemas compartilham subproblemas
 - Um subproblema já resolvido é reutilizado para resolver outro
- Difere do método de Divisão e Conquista
 - Subproblemas não são reaproveitados
 - Este resolve o mesmo problema repetidamente

Programação Dinâmica

- Resolve cada subproblema só uma vez
- Grava a solução do subproblema numa tabela
- Evita o retrabalho
- Constrói por etapas uma resposta combinando respostas já obtidas para partes menores

Programação Dinâmica

- Em geral, é aplicada a problemas de otimização
 - Podem ter muitas soluções possíveis
 - Desejamos encontrar uma solução ótima
 - Podem haver várias soluções ótimas

Programação Dinâmica

- O desenvolvimento de um algoritmo de PD segue 4 etapas:
 - Caracterizar a estrutura de uma solução ótima
 - Definir recursivamente o valor de uma solução ótima
 - Calcular o valor de uma solução ótima
 - Construir uma solução ótima com as informações calculadas

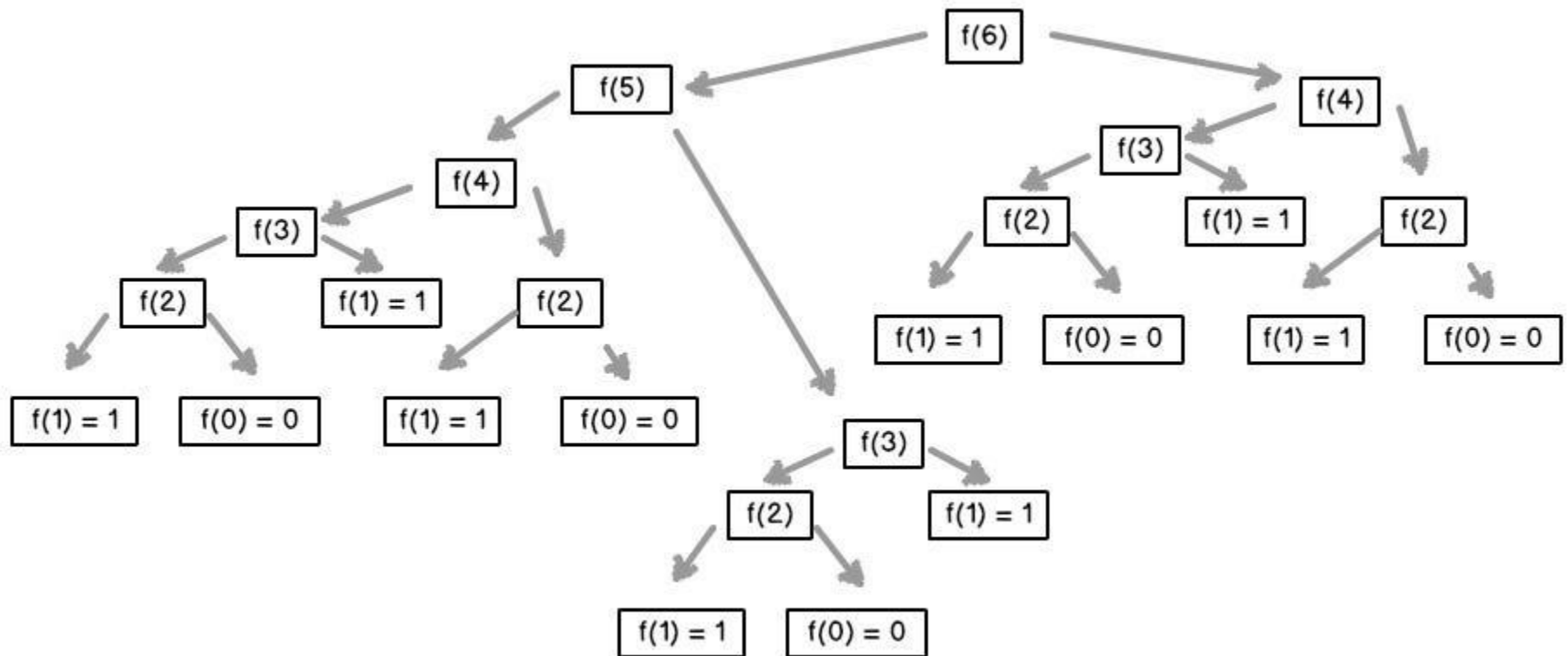
Exemplo

- Para encontrar o termo n da sequência de Fibonacci, em geral fazemos:

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n-1) + fib(n-2)
```

Exemplo

- Para encontrar o termo n da sequência de Fibonacci, em geral fazemos:
- $f(3)$ é recalculado 3 vezes, $f(2)$ 5x



Exemplo

- Para encontrar o termo n da sequência de Fibonacci, usando programação dinâmica:

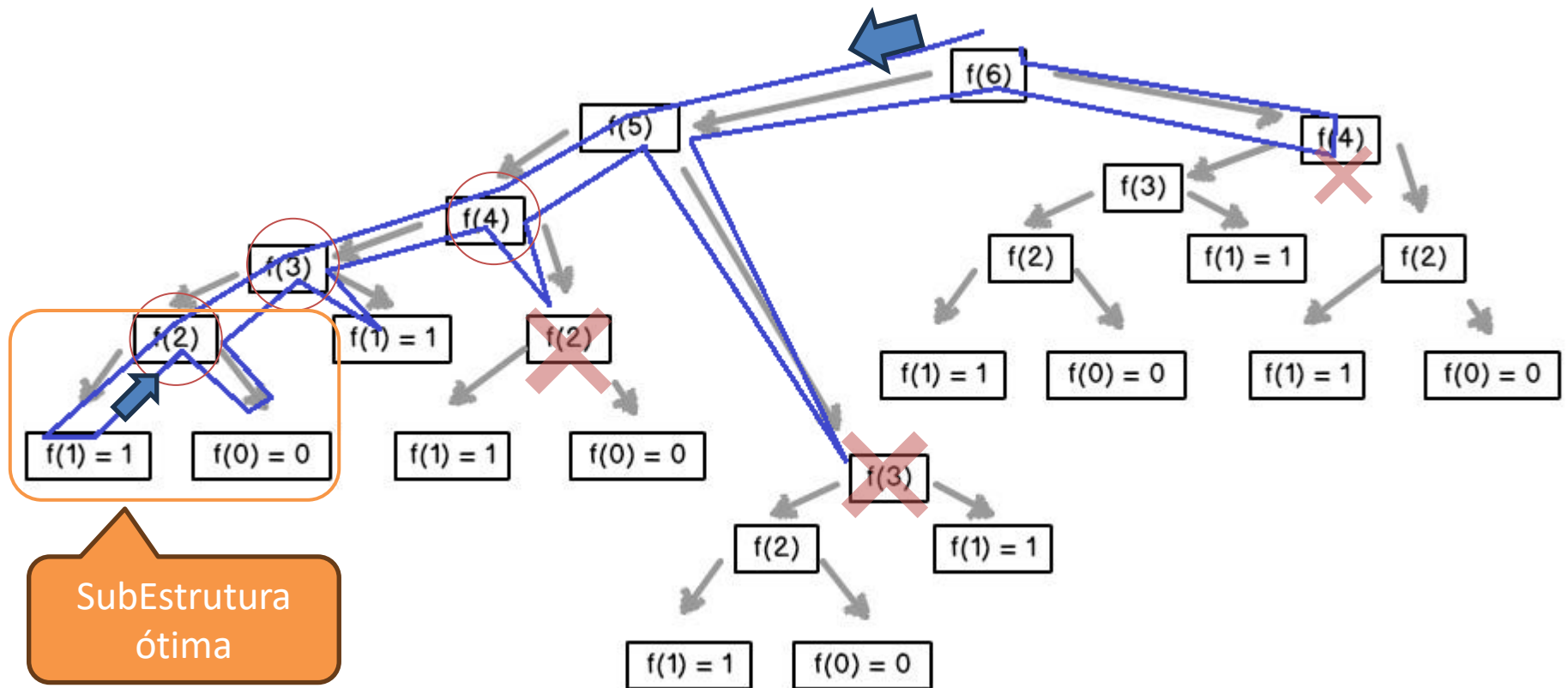
Mapa de valores já
calculados

```
fib_mapa = {0:0, 1:1}
def fib_pd(n):
    if not n in fib_mapa:
        fib_mapa[n] = fib_pd(n - 1) + fib_pd(n - 2)
    return fib_mapa[n]
```

Só calcula se não
estiver no mapa

Exemplo

- Reaproveitamento de sub-soluções



Exemplo

- Executando `fib(36)` e `fib_pd(36)`:

```
fib: 14930352
```

```
Tempo: 2.861
```

```
fib_pd: 14930352
```

```
Tempo: 0.000
```

Exemplo

- Executando fib_pd(800):

```
fib_pd: 6928308186422471713629007768132851
827339912438520482071896604059769143558727
838311227716196753253067537417085740474301
762346722036177801617210685583897575998519
0398725
Tempo: 0.000
```